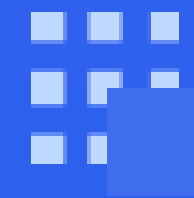




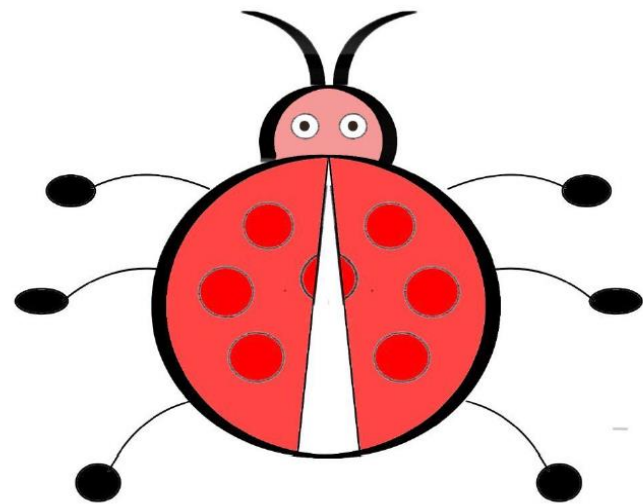
软件测试

Bug的不确定性



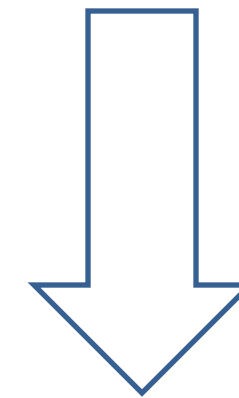
Bug 的反向定义

给定程序 P 和测试 t , 若 $P(t)$ 失效, 修改后得到新程序 P' , 若 $P'(t)$ 通过, 则确认程序修复位置的源代码 $P \setminus P'$ 为故障。



Bug 的不确定性

给定待测程序 P 和测试 t , $P(t)$ 失效。若不同修复方法分别得到两个程序 $P_1 \neq P_2$, 均使得测试 t 通过, 即失效消失, 从而确认了 $P \setminus P_1$ 和 $P \setminus P_2$ 为两个不同的故障。



- Bug 的反向定义性质导致两种现象, 即可能引发错误的程序修复。
- Bug 的反向定义性质导致两种现象, 可能导致多种不同的正确修复。

深入分析Bug ■ Bug的不确定性

```
int MAX1(int x, int y) {  
    int mx = x;  
    if (x > y) {  
        mx = y;  
    }  
    return mx;  
}
```

- 左边是计算两个输入值的极大值函数 MAX1。这个函数接受两个整数 x 和 y 作为 输入，并返回它们中的较小值。
- 它将 x 的值赋给变量 mx，然后检查 x 是否大于 y。如果是，则将 y 的值赋给 mx。最后，函数返回 mx。
- 在 MAX1 示例程序中，输入两个整数，输出大的那个数字。当输入“3,5”，程序输出了“3”，跟预期输出“5” 不符合，产生了 failure-失效。
- 程序员进行测试和调试分析。由于程序员的不同习惯，关于 Fault-故障的理解往往有所不同。

深入分析Bug ■ Bug的不确定性

```
int MAX1(int x, int y) {  
    int mx = x;  
    if (x > y) {  
        mx = y;  
    }  
    return mx;  
}
```

```
int MAX2(int x, int y) {  
    int mx = x;  
    if (x < y) { // 修复了 x > y  
        mx = y;  
    }  
    return mx;  
}
```

- 程序员A审查MAX1程序时，怀疑第3行代码的不等式搞反了。
- 因此尝试把第3行代码修改成 “x < y”，得到新程序MAX2。
- 针对MAX2重新运行测试 “3,5”，此时输出了“5”，测试通过。
- 因此程序员A断定第3行代码是Fault-故障。

```
int MAX1(int x, int y) {  
    int mx = x;  
    if (x > y) {  
        mx = y;  
    }  
    return mx;  
}
```

```
int MAX3(int x, int y){  
    int mx = y; // 修复了mx=x;  
    if (x>y){  
        mx = x; // 修复了mx=y  
    }  
    return mx;  
}
```

- 程序员B审查MAX2程序时，怀疑第2行代码和第4行代码的赋值搞反了。
- 因此尝试把第2行代码和第4行代码进行了调换，得到新程序MAX3。
- 针对MAX3重新运行测试“3,5”，此时输出了“5”，测试通过。
- 因此程序员B断定第4行代码是Fault-故障。

深入分析Bug ■ Bug的不确定性

```
int MAX1(int x, int y) {  
    int mx = x;  
    if (x > y) {  
        mx = y;  
    }  
    return mx;  
}
```

```
int MAX2(int x, int y) {  
    int mx = x;  
    if (x < y) { // 修复了 x > y  
        mx = y;  
    }  
    return mx;  
}
```

```
int MAX3(int x, int y) {  
    int mx = y; // 修复了 mx = x;  
    if (x > y) {  
        mx = x; // 修复了 mx = y  
    }  
    return mx;  
}
```

- 程序员A认为第三行代码是Bug。
- 程序员B认为第二和第四行代码是Bug。
- 两个语法不等的程序：程序Max2和Max3是语义等价的。

- 对于任意一个程序，能够构造无穷多个跟它语义相等但语法不同的程序。这也意味着，能够定义无穷多个Bug，这导致程序修复收敛具有很大的不确定性。
- 为了降低这种不确定性，需要更多的工程化方法，例如要求极小化的程序修复，即尽可能少修改程序代码。
- 对于任意的需求规格，理论上都存在无穷的正确程序实现版本，这些程序版本是语法不一致的。这样的编程特性，给后期测试、确定 Bug、进而调试和修复带来极大的挑战。

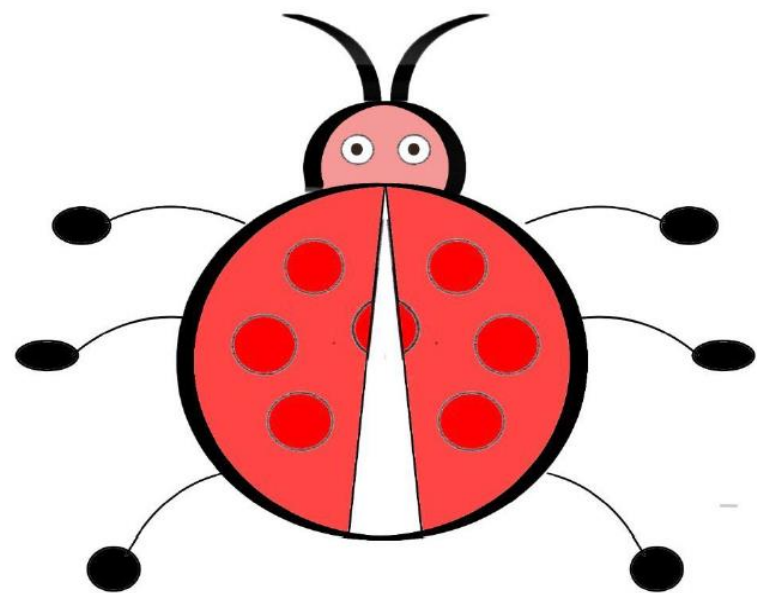
```
void Mean1(double X[], int L) {  
    double mean, sum;  
    sum = 0;  
    for (int i = 1; i < L; i++) {  
        sum += X[i];  
    }  
    mean = sum / L;  
    printf("Mean: %f\n", mean);  
}
```

- 给定左边的程序 P
- 给定测试 $t=[4,3,5]$, 预期输出4
- 执行测试 t , $P(t)$ 输出2.67

思考与讨论:

1. 如何修改程序 P , 使得测试 t 通过?
2. 存在几种修改方法使得测试 t 通过?

回顾与讨论



Bug的反向定义

- 不同修复方法带来不同的Bug
- 不同测试确认带来不同的Bug

先有一个完全正确的程序，然后注入Bug进行讲解和分析。

面对一个存在众多Bug的程序，然后逐步定位修复和测试确认。

教学
科研

应用
实践